

Query Processing Notes for DBMS Placements

Query processing is the journey of an SQL query from text written by the user to actual data retrieval from disk or memory. For placements, you should be able to explain the major phases, why optimization is needed, how execution plans work, and how common join algorithms are chosen.

1. Big Picture

When we write:

```
SELECT e.name, d.name
FROM employees e
JOIN departments d ON e.dept_id = d.id
WHERE e.salary > 50000;
```

the DBMS does not simply execute the query line by line. It transforms the query into an internal representation, checks correctness, explores alternative ways to execute it, estimates costs, picks a good plan, and finally runs that plan.

High-level flow:

```
SQL query
  |
  v
Parsing
  |
  v
Semantic analysis
  |
  v
Query rewrite / logical optimization
  |
  v
Physical optimization
  |
  v
Execution plan
  |
  v
Query execution
  |
  v
Result
```

Main Phases

Phase	Meaning
Parsing	Checks SQL syntax and builds a parse tree
Semantic analysis	Checks table names, column names, data types, privileges
Query rewrite	Converts query into equivalent but simpler or better logical form
Optimization	Chooses an efficient execution strategy
Plan generation	Produces a physical execution plan
Execution	Runs operators such as scan, filter, join, sort, aggregate

2. Why Query Processing Matters

Two SQL queries can produce the same result but have very different execution times.

Example:

```
SELECT *  
FROM orders  
WHERE customer_id = 10;
```

If there is no index on `customer_id`, the DBMS may scan the whole `orders` table.

If there is an index on `customer_id`, the DBMS can jump directly to matching rows.

For large data, the difference can be seconds versus milliseconds.

Important DBMS goal:

```
Find a correct execution plan with low estimated cost.
```

The optimizer usually tries to minimize I/O cost because disk access is much slower than CPU and memory operations. Modern optimizers also consider CPU, memory, network, parallelism, and caching.

3. Parsing

Parsing checks whether the SQL query follows the grammar of SQL.

Example of syntactically valid query:

```
SELECT name  
FROM employees  
WHERE salary > 50000;
```

Example of syntax error:

```
SELECT name FROM WHERE employees;
```

The parser produces a parse tree or abstract syntax tree.

For:

```
SELECT name
FROM employees
WHERE salary > 50000;
```

the tree roughly represents:

```
SELECT
  columns: name
  FROM: employees
  WHERE:
    >
    left: salary
    right: 50000
```

Lexical Analysis

Before parsing, the query is broken into tokens.

For:

```
SELECT name FROM employees WHERE salary > 50000;
```

tokens are:

```
SELECT, name, FROM, employees, WHERE, salary, >, 50000, ;
```

Syntax Analysis

Syntax analysis checks whether tokens follow SQL grammar.

For example, SQL expects:

```
SELECT <select-list> FROM <table-list> WHERE <condition>
```

Parse Tree vs Query Tree

Term	Meaning
Parse tree	Direct representation of SQL syntax
Query tree	More meaningful internal representation using relational algebra operators

The parse tree is mostly about grammar. The query tree is closer to how the DBMS reasons about the query.

4. Semantic Analysis

After parsing, the DBMS checks whether the query is meaningful.

Semantic checks include:

- Does the table exist?
- Does the column exist?
- Is the column reference ambiguous?
- Are the data types compatible?
- Does the user have permission?
- Is the aggregate usage valid?
- Are **GROUP BY** rules followed?

Example:

```
SELECT age
FROM employees;
```

If **employees** has no column named **age**, syntax is correct but semantic analysis fails.

Example of ambiguous column:

```
SELECT id
FROM employees e
JOIN departments d ON e.dept_id = d.id;
```

If both tables have **id**, the DBMS may require:

```
SELECT e.id
FROM employees e
JOIN departments d ON e.dept_id = d.id;
```

5. Relational Algebra Translation

SQL is declarative: we say what result we want.

Relational algebra is procedural/logical: it describes operations such as selection, projection, join, and grouping.

Common relational algebra operators:

Operator	Symbol	SQL Equivalent
Selection	σ	<code>WHERE</code>
Projection	π	<code>SELECT columns</code>
Join	$\bowtie$	<code>JOIN</code>
Cartesian product	$\times$	<code>FROM A, B</code> without join condition
Union	$\cup$	<code>UNION</code>
Difference	$-$	<code>EXCEPT / MINUS</code>
Grouping	γ	<code>GROUP BY</code>

Example SQL:

```
SELECT name
FROM employees
WHERE salary > 50000;
```

Relational algebra:

```
 $\pi_{name} ( \sigma_{salary > 50000} (employees) )$ 
```

For join:

```
SELECT e.name, d.name
FROM employees e
JOIN departments d ON e.dept_id = d.id;
```

Relational algebra:

```
 $\pi_{e.name, d.name} ( employees \bowtie_{e.dept_id = d.id} departments )$ 
```

6. Query Optimization

Query optimization is the process of choosing an efficient plan among many equivalent plans.

The optimizer answers questions like:

- Which table should be scanned first?
- Should the DBMS use an index or full table scan?
- In what order should joins be performed?
- Which join algorithm should be used?
- Should sorting be done now or later?
- Can filters be pushed closer to base tables?
- Can projections remove unnecessary columns early?

Logical Optimization

Logical optimization rewrites the query into an equivalent relational algebra expression.

Important logical rules:

1. Push selections down.
2. Push projections down.
3. Replace Cartesian product plus condition with join.
4. Reorder joins.
5. Simplify boolean conditions.
6. Remove redundant operations.
7. Convert subqueries to joins where useful.

Physical Optimization

Physical optimization chooses actual algorithms and access paths.

Examples:

Logical Operation	Possible Physical Implementations
Table access	Sequential scan, index scan
Selection	Scan filter, index lookup
Join	Nested loop join, index nested loop join, hash join, sort-merge join
Sorting	In-memory sort, external merge sort
Aggregation	Sort-based aggregation, hash-based aggregation

7. Selection Pushdown

Selection pushdown means applying filters as early as possible.

Bad logical plan:

```
Join employees and departments
Then filter salary > 50000
```

Better logical plan:

```
Filter employees where salary > 50000  
Then join with departments
```

Why better?

Filtering early reduces the number of rows involved in later operations.

Example:

```
SELECT e.name, d.name  
FROM employees e  
JOIN departments d ON e.dept_id = d.id  
WHERE e.salary > 50000;
```

Optimized idea:

```
(sigma_salary > 50000 employees) join departments
```

8. Projection Pushdown

Projection pushdown means keeping only required columns as early as possible.

If the final output needs only `e.name` and `d.name`, the DBMS may avoid carrying unnecessary columns like `address`, `phone`, `date_of_birth`, etc.

Why useful?

- Reduces memory usage
- Reduces disk I/O
- Reduces network transfer in distributed DBMS
- Makes joins and sorts cheaper

9. Join Ordering

Join order heavily affects performance.

For:

```
SELECT *  
FROM A  
JOIN B ON A.x = B.x  
JOIN C ON B.y = C.y;
```

possible orders include:

```
(A join B) join C
A join (B join C)
(A join C) join B, if valid through conditions
```

If **A** has 1 million rows, **B** has 100 rows, and **C** has 50 rows, joining smaller filtered tables first may be much cheaper.

Left-Deep, Right-Deep, and Bushy Trees

Left-deep plan:

```
((A join B) join C) join D
```

Right-deep plan:

```
A join (B join (C join D))
```

Bushy plan:

```
(A join B) join (C join D)
```

Many optimizers prefer left-deep plans because they are simpler to enumerate and execute using pipelining.

10. Execution Plan

An execution plan is the actual physical plan chosen by the optimizer.

It specifies:

- Access methods
- Join order
- Join algorithms
- Sort operations
- Aggregate algorithms
- Estimated number of rows
- Estimated cost
- Sometimes actual runtime statistics

Example plan shape:

```
Hash Join
  Hash Cond: e.dept_id = d.id
    -> Seq Scan on employees e
```



```
Filter: salary > 50000
-> Hash
    -> Seq Scan on departments d
```

Meaning:

1. Scan **employees**.
2. Filter employees with salary greater than 50000.
3. Scan **departments**.
4. Build a hash table on departments.
5. Probe hash table using **employees.dept_id**.

Logical Plan vs Physical Plan

Logical Plan	Physical Plan
Describes what operations are needed	Describes how operations are executed
Uses relational algebra	Uses concrete algorithms
Example: join A and B	Example: hash join A and B
Independent of storage details	Depends on indexes, statistics, memory, data size

EXPLAIN

Many DBMSs provide an **EXPLAIN** command.

Example:

```
EXPLAIN
SELECT e.name, d.name
FROM employees e
JOIN departments d ON e.dept_id = d.id
WHERE e.salary > 50000;
```

EXPLAIN shows the estimated plan.

Some DBMSs support runtime execution details:

```
EXPLAIN ANALYZE
SELECT ...
```

EXPLAIN ANALYZE actually runs the query and shows estimated plus actual values.

11. Access Methods

Access method means how the DBMS reads rows from a table.

Sequential Scan

Sequential scan reads the whole table.

Useful when:

- Table is small
- Most rows are needed
- No useful index exists
- Reading sequentially is cheaper than many random index lookups

Cost rough idea:

```
Cost = number of blocks in table
```

If table has $B(R)$ blocks:

```
Seq scan cost =  $B(R)$ 
```

Index Scan

Index scan uses an index to locate matching rows.

Useful when:

- Predicate is selective
- Query returns small percentage of rows
- Index exists on filtered or joined column

Example:

```
SELECT *  
FROM employees  
WHERE emp_id = 101;
```

With an index on `emp_id`, the DBMS can find the row quickly.

Index-Only Scan

An index-only scan can answer the query using only the index, without fetching table rows.

Example:

```
SELECT emp_id  
FROM employees
```

```
WHERE emp_id BETWEEN 100 AND 200;
```

If `emp_id` is indexed and no other column is needed, table access may be avoided.

Clustered vs Non-Clustered Index Impact

Index Type	Effect
Clustered index	Data rows are physically ordered by index key
Non-clustered index	Index points to rows stored elsewhere

Range queries are often faster with clustered indexes because matching rows are physically close.

12. Join Algorithms

A join combines rows from two relations based on a condition.

Common join algorithms:

1. Nested loop join
2. Block nested loop join
3. Index nested loop join
4. Hash join
5. Sort-merge join

Important notation:

Symbol	Meaning
R, S	Relations/tables
M, N	Number of tuples/rows in R and S
$B(R), B(S)$	Number of disk blocks/pages in R and S
$V(A, R)$	Number of distinct values of attribute A in R
M memory blocks	Number of available buffer pages

13. Nested Loop Join

Nested loop join compares each row of one table with rows of another table.

Basic idea:

```
for each row r in R:
  for each row s in S:
    if join_condition(r, s):
      output r combined with s
```

SQL example:

```
SELECT *  
FROM R  
JOIN S ON R.a = S.b;
```

Simple Nested Loop Join Cost

If R is outer and S is inner:

$$\text{Cost} = B(R) + T(R) * B(S)$$

where:

- $B(R)$ = blocks of outer relation
- $T(R)$ = tuples of outer relation
- $B(S)$ = blocks of inner relation

This can be very expensive because the inner relation is scanned again for every tuple of the outer relation.

If both tables have many rows, simple nested loop join is usually bad.

When Nested Loop Join Is Useful

Nested loop join is useful when:

- Outer relation is very small
- Inner relation has an index on join column
- Join condition is non-equi join like $<$, $>$, **BETWEEN**
- Memory is limited

Block Nested Loop Join

Instead of reading one tuple of outer relation at a time, block nested loop join reads a block or group of blocks.

Pseudo-code:

```
for each block Br of R:  
    for each block Bs of S:  
        compare tuples in Br with tuples in Bs
```

Cost:

$$\text{Cost} = B(R) + B(R) * B(S)$$

If multiple buffer pages are available:

$$\text{Cost} = B(R) + \text{ceil}(B(R) / (M - 2)) * B(S)$$

where M is number of available memory buffers.

$M - 2$ because one buffer is needed for inner relation and one for output.

Choosing Outer Relation

For nested loop joins, the smaller relation is usually chosen as the outer relation to reduce repeated inner scans.

If $B(R) < B(S)$, prefer:

R as outer, S as inner

14. Index Nested Loop Join

Index nested loop join uses an index on the inner relation's join attribute.

Pseudo-code:

```
for each row r in R:
    use index on S.b to find rows where S.b = r.a
    output matches
```

Useful query:

```
SELECT *
FROM employees e
JOIN departments d ON e.dept_id = d.id;
```

If `departments.id` is indexed, for each employee the DBMS can quickly find the matching department.

Rough cost:

$$\text{Cost} = B(R) + T(R) * \text{index_lookup_cost}$$

Index lookup cost depends on:

- B+ tree height
- Whether index is clustered
- Number of matching rows
- Random I/O cost

Good Cases

Index nested loop join works well when:

- Outer table is small
- Inner join attribute is indexed
- Join is selective
- Index lookup returns few rows per outer row

Bad Cases

It can be poor when:

- Outer table is huge
- Many index lookups are needed
- Each lookup causes random I/O
- Many rows match each lookup

15. Hash Join

Hash join is commonly used for equi-joins.

Example:

```
SELECT *  
FROM R  
JOIN S ON R.a = S.b;
```

Hash join has two phases:

1. Build phase
2. Probe phase

Build Phase

The DBMS chooses the smaller relation, say **R**, and builds a hash table on the join attribute.

```
hash_table = {}  
for each row r in R:  
    hash_table[hash(r.a)].add(r)
```

Probe Phase

The DBMS scans the other relation, say S , and probes the hash table.

```
for each row  $s$  in  $S$ :  
    bucket = hash_table[hash( $s.b$ )]  
    compare  $s$  with rows in bucket  
    output matches
```

Hash Join Cost

If the smaller relation fits in memory:

$$\text{Cost} = B(R) + B(S)$$

This is very efficient because both tables are scanned once.

If data does not fit in memory, the DBMS uses partitioned hash join.

Partitioned Hash Join

Steps:

1. Partition both relations using a hash function.
2. Matching rows must fall into corresponding partitions.
3. Join each pair of partitions separately.

Rough cost:

$$\text{Cost} = 3 * (B(R) + B(S))$$

Why 3?

- Read R and S for partitioning
- Write partitions
- Read partitions again for joining

Advantages of Hash Join

- Very fast for large equi-joins
- Does not require sorted input
- Works well when one relation fits in memory
- Often better than nested loops for large tables

Limitations of Hash Join

- Mainly useful for equi-joins
- Not suitable for range joins like $<$, $>$, **BETWEEN**
- Needs memory for hash table
- Can degrade with hash collisions or skewed data
- May spill to disk if memory is insufficient

Data Skew Problem

Data skew means some join key values appear much more often than others.

Example:

```
dept_id = 10 appears in 70% of employees
```

This can create very large hash buckets and slow down the join.

16. Sort-Merge Join

Sort-merge join is also used for equi-joins and range-style ordered processing.

It has two main phases:

1. Sort both relations on join attribute.
2. Merge sorted relations to find matches.

Example:

```
SELECT *  
FROM R  
JOIN S ON R.a = S.b;
```

Steps:

```
sort R on a  
sort S on b  
merge the sorted results
```

Merge Phase

If both inputs are sorted:

```
i = first row of R  
j = first row of S  
  
while i and j exist:
```



```
if R[i].a < S[j].b:
    advance i
else if R[i].a > S[j].b:
    advance j
else:
    output all matching rows
    advance through matching group
```

Sort-Merge Join Cost

If both relations are already sorted:

$$\text{Cost} = B(R) + B(S)$$

If sorting is needed:

$$\text{Cost} = \text{sort}(R) + \text{sort}(S) + B(R) + B(S)$$

External sorting cost is often:

$$\text{Cost} = 2 * B * \text{number_of_passes}$$

When Sort-Merge Join Is Useful

Sort-merge join is useful when:

- Both inputs are already sorted
- Indexes provide sorted order
- Output is required in sorted order
- Join is large
- Memory is not enough for hash join
- Join condition benefits from ordering

Limitations

- Sorting can be expensive
- Usually slower than hash join if unsorted inputs and enough memory for hashing
- Handling duplicates requires care

17. Join Algorithm Comparison

Join Algorithm	Best For	Main Requirement	Weakness
Simple nested loop	Small tables	None	Very expensive for large tables
Block nested loop	Medium tables with limited memory	Buffer pages	Repeated scans of inner table
Index nested loop	Small outer table + indexed inner table	Index on inner join column	Many random I/Os if outer is large
Hash join	Large equi-joins	Hashable equality condition	Needs memory, not good for non-equi joins
Sort-merge join	Sorted inputs, large joins, ordered output	Sorted inputs or sort step	Sorting cost can be high

Placement shortcut:

```

Small outer + index on inner -> Index nested loop join
Large equi-join -> Hash join
Already sorted inputs -> Sort-merge join
No index and small relation -> Block nested loop join
Non-equi join -> Nested loop or sort-based strategy

```

18. Cost Estimation

Cost estimation predicts the resource usage of a query plan.

The optimizer estimates:

- Number of rows produced by each operator
- Number of disk pages read or written
- CPU work
- Memory usage
- Network cost in distributed DBMS
- Sorting and hashing cost

Why Cost Estimation Is Hard

The optimizer does not know exact intermediate result sizes before executing the query. It relies on statistics.

Statistics include:

- Table cardinality: number of rows
- Number of pages/blocks
- Number of distinct values per column
- Min and max values
- Histograms
- Null fraction

- Most common values
- Index statistics

Cardinality

Cardinality means number of tuples/rows.

If:

```
T(employees) = 100000
```

then employees has 100000 rows.

Selectivity

Selectivity is the fraction of rows that satisfy a condition.

```
selectivity = matching rows / total rows
```

If 1000 out of 100000 employees have salary greater than 100000:

```
selectivity = 1000 / 100000 = 0.01
```

Estimated result rows:

```
output rows = input rows * selectivity
```

Equality Predicate Estimation

For:

```
WHERE dept_id = 10
```

If `dept_id` has $V(\text{dept_id}, \text{employees})$ distinct values and values are uniformly distributed:

```
selectivity = 1 / V(dept_id, employees)
```

Estimated rows:

$$T(\text{employees}) / V(\text{dept_id}, \text{employees})$$

Example:

```
T(employees) = 100000
V(dept_id, employees) = 100

Estimated rows = 100000 / 100 = 1000
```

Range Predicate Estimation

For:

```
WHERE salary > 50000
```

If min salary is 10000 and max salary is 110000, assuming uniform distribution:

```
selectivity = (110000 - 50000) / (110000 - 10000)
             = 60000 / 100000
             = 0.6
```

Estimated rows:

$$T(\text{employees}) * 0.6$$

Real systems use histograms because values are rarely uniform.

Join Cardinality Estimation

For equi-join:

```
R JOIN S ON R.a = S.b
```

Common estimate:

$$T(R \text{ join } S) = T(R) * T(S) / \max(V(a, R), V(b, S))$$

Example:

```
T(R) = 10000
T(S) = 5000
V(a, R) = 100
V(b, S) = 200

Estimated join rows = 10000 * 5000 / max(100, 200)
                    = 50000000 / 200
                    = 250000
```

Cost Units

Cost is usually an internal unit, not seconds.

A plan with cost **100** is estimated to be cheaper than cost **1000**, but it does not necessarily mean 100 ms or 100 disk reads.

19. Important Cost Formulas

Let:

```
B(R) = number of blocks in R
B(S) = number of blocks in S
T(R) = number of tuples in R
T(S) = number of tuples in S
M = number of memory buffer blocks
```

Sequential Scan

```
Cost = B(R)
```

Simple Nested Loop Join

R as outer, S as inner:

```
Cost = B(R) + T(R) * B(S)
```

Page-Oriented Nested Loop Join

```
Cost = B(R) + B(R) * B(S)
```

Block Nested Loop Join

$$\text{Cost} = B(R) + \text{ceil}(B(R) / (M - 2)) * B(S)$$

Index Nested Loop Join

$$\text{Cost} = B(R) + T(R) * \text{cost_to_find_matching_S_rows}$$

In-Memory Hash Join

$$\text{Cost} = B(R) + B(S)$$

Grace / Partitioned Hash Join

$$\text{Cost} = 3 * (B(R) + B(S))$$

Sort-Merge Join

If inputs are sorted:

$$\text{Cost} = B(R) + B(S)$$

If not sorted:

$$\text{Cost} = \text{sort}(R) + \text{sort}(S) + B(R) + B(S)$$

External Merge Sort

Approximate:

$$\text{Cost} = 2 * B * \text{number_of_passes}$$

Number of initial runs:

$$\text{ceil}(B / M)$$

Each merge pass can merge roughly $M - 1$ runs.

20. External Sorting

External sorting is used when data does not fit in memory.

DBMS typically uses external merge sort.

Phase 1: Run Generation

1. Read M blocks into memory.
2. Sort them in memory.
3. Write sorted run to disk.
4. Repeat for all blocks.

Number of runs:

$$\text{ceil}(B / M)$$

Phase 2: Merge Runs

Merge multiple sorted runs into larger sorted runs.

If memory has M buffers, DBMS can merge at most $M - 1$ runs at a time, keeping one output buffer.

Cost

Each pass reads and writes all blocks:

$$\text{Cost per pass} = 2B$$

Total cost:

$$2B * \text{number_of_passes}$$

21. Pipelining vs Materialization

Pipelining

Pipelining passes output of one operator directly to the next operator without storing the full intermediate result.

Example:

```
Seq Scan -> Filter -> Projection
```

Advantages:

- Saves memory
- Avoids writing intermediate results to disk
- Can start producing output earlier

Materialization

Materialization stores intermediate results before the next operator consumes them.

Example:

```
Compute subquery result
Store it
Then join with another table
```

Advantages:

- Useful when result is reused
- Useful for blocking operators
- Simplifies execution

Blocking Operators

Blocking operators must consume most or all input before producing output.

Examples:

- Sort
- Group by with sort
- Duplicate elimination
- Some aggregation operations

Non-blocking operators can produce output row by row.

Examples:

- Selection
- Projection
- Nested loop join in some cases

22. Query Rewrite Examples

Convert Subquery to Join

Original:

```
SELECT name
FROM employees
WHERE dept_id IN (
```



```
SELECT id
FROM departments
WHERE location = 'Delhi'
);
```

Possible rewrite:

```
SELECT e.name
FROM employees e
JOIN departments d ON e.dept_id = d.id
WHERE d.location = 'Delhi';
```

Predicate Simplification

Original:

```
WHERE salary > 50000 AND salary > 70000
```

Simplified:

```
WHERE salary > 70000
```

Remove Redundant Projection

If a query already needs only specific columns, intermediate columns can be removed.

23. Heuristic Optimization

Heuristic optimization uses rules that are usually good.

Common heuristics:

1. Perform selection early.
2. Perform projection early.
3. Avoid Cartesian products.
4. Join smaller relations first.
5. Use indexes for selective predicates.
6. Replace repeated subquery evaluation with joins where possible.

Heuristic optimization is fast but may miss the best plan.

24. Cost-Based Optimization

Cost-based optimization estimates costs for multiple plans and chooses the cheapest estimated plan.

Steps:

1. Generate candidate logical plans.
2. Generate physical plans for each logical plan.
3. Estimate cardinality of intermediate results.
4. Estimate I/O, CPU, memory, and network costs.
5. Choose plan with minimum estimated cost.

Dynamic Programming in Join Optimization

For multiple joins, optimizers often use dynamic programming.

For tables **A**, **B**, **C**, **D**, the optimizer finds best plans for:

```
Single tables
Pairs of tables
Triples of tables
All four tables
```

This avoids blindly enumerating every possible full plan.

Search Space Problem

Join order possibilities grow very quickly.

For many tables, exhaustive optimization becomes expensive, so optimizers may use:

- Heuristics
- Greedy algorithms
- Genetic algorithms
- Pruning
- Time limits

25. Statistics and Histograms

The optimizer relies heavily on statistics.

If statistics are outdated, the optimizer may choose a bad plan.

Example:

```
WHERE status = 'FAILED'
```

If the optimizer thinks only 1% rows have status **FAILED**, it may use an index.

If actually 80% rows are **FAILED**, a sequential scan may be better.

Histograms

Histograms store approximate distribution of column values.

They help estimate:

- Range predicates
- Skewed data
- Popular values

Without histograms, optimizer often assumes uniform distribution, which can be wrong.

26. Interesting Orders

An interesting order is a sort order that may be useful later in the query.

Example:

```
SELECT dept_id, COUNT(*)  
FROM employees  
GROUP BY dept_id  
ORDER BY dept_id;
```

If data is already sorted by `dept_id`, grouping and ordering may become cheaper.

Sort-merge join can also produce sorted output, which may help an `ORDER BY` later.

27. Aggregation Execution

SQL aggregation:

```
SELECT dept_id, COUNT(*), AVG(salary)  
FROM employees  
GROUP BY dept_id;
```

Common physical strategies:

Hash Aggregation

Build a hash table where key is group-by column.

```
hash[dept_id].count += 1  
hash[dept_id].sum_salary += salary
```

Good when groups fit in memory.

Sort Aggregation

Sort rows by group-by columns, then scan sorted data and aggregate consecutive rows.

Good when:

- Data is already sorted
- Output needs to be sorted
- Hash table may not fit in memory

28. Query Execution Engine

The execution engine runs the selected plan.

Each physical operator implements an interface such as:

```
open()  
next()  
close()
```

This is called the iterator model or Volcano model.

Example:

```
Projection  
  Filter  
    Seq Scan employees
```

Execution:

1. Projection asks Filter for next row.
2. Filter asks Seq Scan for next row.
3. Seq Scan reads row.
4. Filter checks condition.
5. Projection returns selected columns.

29. Volcano Iterator Model

Each operator behaves like an iterator.

```
open(): initialize operator  
next(): produce next tuple  
close(): release resources
```

Advantages:

- Modular
- Easy to compose operators
- Supports pipelining

Disadvantages:

- Function call overhead for every tuple
- Less cache-friendly than vectorized execution

30. Vectorized Execution

Modern DBMSs may process batches of rows instead of one row at a time.

Instead of:

```
process one tuple
process next tuple
```

vectorized execution does:

```
process batch of 1024 values
process next batch
```

Advantages:

- Better CPU cache usage
- Less function call overhead
- Better SIMD opportunities

Common in analytical databases.

31. Common Interview Questions

What is query processing?

Query processing is the process by which a DBMS parses, validates, optimizes, and executes an SQL query to produce the required result.

What is query optimization?

Query optimization is the process of selecting an efficient execution plan from many equivalent alternatives.

Difference between heuristic and cost-based optimization?

Heuristic Optimization	Cost-Based Optimization
Uses general rules	Uses estimated costs
Faster	More accurate but more expensive
Example: push selections down	Example: compare hash join cost vs sort-merge join cost
May miss best plan	Usually finds better plan if statistics are accurate

What is an execution plan?

An execution plan is a physical strategy chosen by the DBMS to execute a query. It includes access methods, join order, join algorithms, sorting, aggregation, and estimated costs.

Why is join ordering important?

Join ordering affects intermediate result sizes. A bad join order can create huge temporary results, while a good join order can filter and reduce data early.

When is hash join preferred?

Hash join is preferred for large equi-joins when sufficient memory is available and inputs are not already sorted.

When is sort-merge join preferred?

Sort-merge join is preferred when inputs are already sorted, output needs sorted order, or memory is not suitable for hash join.

When is nested loop join preferred?

Nested loop join is preferred when the outer relation is small, the inner relation has an index, or the join condition is non-equality.

Why can an index scan be slower than a sequential scan?

If a query returns a large portion of the table, index scan may cause many random I/Os. A sequential scan may be cheaper because it reads pages in order.

What are database statistics?

Database statistics are metadata about tables and columns, such as row counts, distinct values, histograms, and index information. Optimizers use them to estimate cost.

What happens if statistics are outdated?

The optimizer may estimate cardinalities incorrectly and choose a poor plan.

32. Quick Revision Tables

Query Processing Pipeline

Step	Purpose
Lexical analysis	Break query into tokens
Parsing	Check grammar and build parse tree
Semantic analysis	Validate names, types, permissions
Query rewrite	Apply equivalent transformations

Step	Purpose
Optimization	Choose efficient logical and physical plan
Execution	Run the selected plan

Operator Types

Operator	Blocking?
Selection	No
Projection	No
Sequential scan	No
Nested loop join	Usually can pipeline
Hash join	Build side is blocking
Sort	Yes
Group by sort	Yes
Duplicate elimination	Usually yes

Access Path Choice

Situation	Likely Access Path
Need most rows	Sequential scan
Need few rows by indexed column	Index scan
Need only indexed columns	Index-only scan
Range query on clustered index	Clustered index scan
Small table	Sequential scan often fine

Join Choice

Situation	Likely Join
Small outer and indexed inner	Index nested loop
Large equi-join	Hash join
Already sorted inputs	Sort-merge join
Need sorted output	Sort-merge join may help
Non-equi join	Nested loop or sort-based approach
Very limited memory	Nested loop may be chosen

33. Worked Example

Tables:

```
employees(emp_id, name, dept_id, salary)
departments(id, name, location)
```

Query:

```
SELECT e.name, d.name
FROM employees e
JOIN departments d ON e.dept_id = d.id
WHERE e.salary > 80000
AND d.location = 'Bangalore';
```

Naive logical plan:

```
Join employees and departments
Then filter salary and location
Then project names
```

Optimized logical plan:

```
Filter employees where salary > 80000
Filter departments where location = 'Bangalore'
Join filtered employees with filtered departments
Project e.name, d.name
```

Possible physical plan:

```
Hash Join on e.dept_id = d.id
-> Seq Scan employees
    Filter salary > 80000
-> Hash
    -> Seq Scan departments
        Filter location = 'Bangalore'
```

If indexes exist:

```
Nested Loop Join
-> Index Scan departments using location index
    location = 'Bangalore'
-> Index Scan employees using dept_id index
```



```
e.dept_id = d.id  
filter salary > 80000
```

Which is better depends on:

- Number of employees
- Number of departments in Bangalore
- Indexes available
- Selectivity of salary predicate
- Memory available
- Accuracy of statistics

34. Common Mistakes in Interviews

Avoid saying:

```
Index is always faster.  
Hash join is always best.  
Optimizer always chooses the perfect plan.  
SQL executes in the order written.  
Nested loop join is always bad.
```

Better answers:

- Indexes help when predicates are selective.
- Hash join is excellent for large equi-joins but needs memory.
- Optimizers choose based on estimates, and estimates can be wrong.
- SQL is declarative; physical execution order is chosen by the optimizer.
- Nested loop join is useful with small outer tables or indexed inner tables.

35. One-Minute Placement Summary

Query processing converts SQL into an efficient execution plan. The DBMS first parses the query, performs semantic checks, translates it into relational algebra, rewrites it using rules like selection pushdown and projection pushdown, then uses a cost-based optimizer to choose access methods, join orders, and physical algorithms. Execution plans use operators such as sequential scan, index scan, nested loop join, hash join, sort-merge join, sort, and aggregation. Cost estimation depends on table statistics, cardinality, selectivity, indexes, and memory. For joins, nested loop is good for small outer relations or indexed inner relations, hash join is strong for large equi-joins, and sort-merge join is useful when inputs are sorted or sorted output is needed.